

FMP: Finite Map Patterns

DEFINITIONS & PROOFS / ENGLISH EDITION

2026-09-23. This edition packages the original **copy n times, leave unavailable maps empty, complete all gaps** rule. It is neither the output-length reindexing nor the deferred one-point-completion system DMP.

- Readable Python kernel; parsing, validation and counts.
- NER registration script: import the complete file into the custom-notation facility of **ne-rewritten**.
- English well-ordering manuscript · 中文证明.

Status: an author-audited **paper proof in ZFC + I3**, not yet Lean-checked or independently refereed. FMP is not included in this repository's weak-KP or Lean-certified collections. The stated sufficient axiom bound is neither an optimal bound nor a lower bound on the notation's order type.

1. Finite syntax

An expression $A = (F_1, \dots, F_N)$ is a finite sequence of columns, numbered from 1. The empty sequence is zero. Each column r is empty or a strictly increasing finite partial map with some marked edges. Every edge $x \mapsto y$ has $x < y$; both sources and targets are strictly increasing. Its last two edges are

$$p_r \mapsto r, \quad e_r \mapsto r + 1, \quad 1 \leq p_r < e_r \leq r.$$

The penultimate source p_r is the pivot; the least source is a_r . Standard expressions are **completed**: $e_r = p_r + 1$. Auxiliary COPY states may have gaps. The point $r + 1$ is a right boundary, not a call to a future column.

Require:

1. Self-closure: every target below p_r is also a source in the same map.
2. Star window: a marked edge $d \mapsto b$ is internal, with $b \geq e_r$.
3. Accurate trace: repeatedly following $u \mapsto p_u$ from b reaches exactly d . An empty column or an overshoot is failure.

The final nonterminal trace column is the bottom column t , so $p_t = d$. Auxiliary uncompleted graphs also use the invariant

$$e_t \leq c, \tag{Q}$$

where c is the next source after this marked edge in its carrier. Completed syntax implies Q automatically: $e_t = d + 1 \leq c$.

Finite syntactic legality is not standard reachability. The standard domain consists only of finite descendants of the seeds in Section 3, not all handwritten legal maps.

2. Lists and marks

The last two edges are recoverable from the column position and pivot:

- `[]`: an empty column.
- `[p]`: only the two endpoint edges.
- `[x:y*,u:v;p]`: internal edges followed by the pivot; an asterisk marks an edge.

For example, `[] [1] [2]` has full tables

```
[][1:2,2:3][2:3,3:4]
```

and count word 1, 2, 3. The 2 in `[2]` is a source address, not count 2. Marks authorize propagation of a composite relation during completion; they cannot be inserted or erased freely. A column is a single-valued

increasing table, with no tree, tensor or merge of competing edges.

3. Seeds, top and standard domain

To construct S_n :

1. Start with an empty column and $\{1 \mapsto 2, 2 \mapsto 3\}$.
2. For $i = 0, \dots, n-1$, put $a = 2i + 1$ and append two columns, with displacements $d = 1, 2$. At new position r , use $\{x \mapsto x + d : a \leq x \leq r - d + 1\}$.
3. All initial star sets are empty.

There are $2 + 2n$ columns, and S_n is a literal prefix of S_{n+1} . For example:

```
S0 = [][1]
S1 = [][1][1:2;2][1:3;2]
```

The display top `Limit` of `FMP` has fundamental-sequence term S_n at index n . It is not an ordinary column graph or a disguised count word. The finite standard domain is the union of the finite descendant sets of these seeds; the display top can be adjoined above it.

4. COPY: one partial address map

For a nonempty last controller at N , write $a = a_N, p = p_N, F = F_N$, and set

$$\phi(x) = \begin{cases} x, & x < a, \\ F(x), & a \leq x < p \text{ and defined,} \\ x + N - p, & x \geq p. \end{cases}$$

Other addresses have no image. Retain the first $N - 1$ columns and append the images of old columns $p, \dots, N$, in order:

- If any required edge source or target has no image, retain the point but make that entire copied map empty.
- Otherwise replace every edge by $\phi(x) \mapsto \phi(y)$.
- Transfer marks only by the following computable rule.

Self-closure makes the controller's own copy always available. One COPY adds $N - p$ columns; this difference is invariant under further COPY operations.

For an old marked edge $d \mapsto b$, take its accurate old trace and let c be its next source. It is eligible to survive in exactly these cases:

1. All nonterminal trace columns are at least p .
2. The first trace column $u < p$ has $u < a$.
3. $a \leq u < p$, the controller has the marked edge $u \mapsto \phi(u)$, and $\phi(c) \leq a$.

Finally require an accurate trace from $\phi(b)$ to $\phi(d)$ in the already output graph. This check cannot be dropped: an intervening copied map may have become empty.

Leaving an unavailable map empty is part of the mathematical definition, not a response to a resource limit. It neither deletes the named point nor replaces the whole child by a cut.

5. COMPLETE: full completion in one scan

Scan rightwards from a specified position, processing only columns present at scan entry and skipping each newly created family. Start with an empty width record H . Already output columns never change.

5.1 Propagate marked packets

At column r , freeze its current old marked edges and process them in target order. For $d \mapsto b$, if its accurate bottom column t has $H(t) = h > 0$, add and mark

$$d + j \mapsto b + j \quad (1 \leq j \leq h).$$

Otherwise do nothing. Newly added marks are not processed again in this event.

Q and the capacity propagated up the accurate trace place each packet in the open intervals between adjacent old sources and targets. Packets do not overlap old edges or one another and do not change a, p, e . This is a structural invariant, not post-hoc deduplication; the proof manuscript supplies the finite closure argument.

5.2 Fill the native gap

Let the final sources be $p < e$ and put $h = e - p - 1$. If $h = 0$, continue. Otherwise:

1. Insert h points after r ; increase all old addresses greater than r by h .
2. Retain all edges except the last cap edge and append $(p + j) \mapsto (r + j) \ (1 \leq j \leq h)$, $e \mapsto (r + h + 1)$.
3. Replace the old column by $h + 1$ restrictions. Restriction j sits at $r + j$ and keeps sources at most $p + j + 1$, for $0 \leq j \leq h$.
4. Keep its old marks and additionally mark targets $r, \dots, r + j - 1$.
5. Record $H(r) = h$ and jump to the next entry-old column.

All output columns have adjacent final sources. Termination of this finite scan does not assume well-foundedness of unbounded repeated expansion.

6. Fundamental sequences

Zero has no mathematical descent edge. The implementation's $\emptyset[n]=\emptyset$ is an interface convention, excluded from descent.

For nonzero A :

- $A[0]$ deletes the last column.
- If the last column is empty, every $A[n]$ deletes it.
- Otherwise, for $n > 0$, perform COPY n times, **delete the actual final controller self-copy in its entirety, then COMPLETE**, starting at the original last position $|A|$.

The order is essential. The index counts copies, not output columns; completion can increase length nonlinearly.

The original first $|A| - 1$ columns are fixed, and

$$A[n] \text{ is a literal complete-column prefix of } A[n + 1].$$

This includes the marks, not just the count word. The next COPY changes only the previous final controller self-copy; after cutting it, raw outputs are nested prefixes. Forward completion is itself prefix-compatible.

7. Local counts and comparison

With the left context fixed, expand a column at a positive index, cut off the tail to its right, and repeat until its position is deleted. This gives its local count: an empty column has count 1, and the first replacement of a nonempty column decreases the count by 1.

The exact shortcut omits marks and the current head address. The empty state is $\perp$; a nonempty state is (p, E) with internal edges E only.

1. Read the state of context column p ; if empty, return $\perp$.
2. If E is empty, return the state just read.
3. Otherwise put $a = \min \text{dom } E$. Define $\psi(x) = x$ below a and $\psi(x) = E(x)$ at the remaining defined sources.
4. If the source state is (q, G) , transport q and all endpoints of G . When $q + 1 < p$, also check the omitted cap source $q + 1$. Any missing image returns $\perp$; otherwise return the transported state.

Give $\perp$ count 1 and accumulate backwards. Memoization is valid only within the same context. In particular,

$$\text{count}([p]) = 1 + \text{count}(F_p).$$

On the standard domain, termination, unique count representation and ordinary lexicographic comparison follow from the paper well-ordering proof and prefix-barrier lemma. This is not a standardness test for arbitrary handwritten maps.

NER locates the earliest differing column and compares the two local trajectories, often stopping when they meet. Unknown counts are never numerical comparison values. Display retains calculated digits and uses ? afterwards. A decrease in count-word lexicographic order alone does not prove well-ordering: lexicographic order on all finite natural-number words is not well-founded.

8. Examples and growth warning

$S_0 = [][1]$ has $n + 1$ empty columns in its n th term. Thus the count word 1, 2 denotes ω , with the indexing convention $\omega[0] = 1$.

Let $A = S_1[0][1] = [][1][2]$, with count word 1, 2, 3. Then

$$|A[n]| = 2^{n+1}, \quad \text{count}(A[n]_r) = 2^{s_2(r-1)},$$

where s_2 counts binary ones. Induct on COPY rounds: completion doubles the block length and doubles counts in the new half. The count of seed column r is $\lfloor (r + 1)^2 / 4 \rfloor$; the local displacement-1 and displacement-2 recurrences give consecutive products and squares.

NER also starts with the examples 1, 2, 2, 4, 2, 4, 4 and 1, 2, 2, 4, 2, 4, 4, 4. The separate comparison paper embeds the whole standard BMS order below the eight-column example, proving that it is at least the BMS limit. Equality and minimality are not established.

Full completion does not meet the lower-priority preference for mild length growth. Counts, numbers of columns and ordinal values must not be conflated.

9. Implementation and proof boundaries

The mathematical kernel uses finite tables, arbitrary-precision Python integers and an optional check callback, without a semantic oracle. JavaScript addresses are safety-checked Numbers; counts are BigInts. Resource exhaustion throws an error, never a truncated substitute fundamental-sequence term.

NER retains five views: compact list, count sequence, full tables, textual adjacency table and triangular adjacency diagram. Diagonal cells serve as row/column labels, with counts above; oversized diagrams are not partially drawn.

FS, FS_alter and FS_short are identical. Default expansion guards are about 450 ms, 1600 columns and 180000 edges; counting has about 70 ms. These are implementation budgets, not mathematical rules or evidence of termination/nontermination.

See VALIDATION for checks. Finite tests validate implementations, not infinite-domain theorems.